

① INTRODUCTION To SYSTEM STRUCTURE

- OS provides an environment for execution of programs & services to programs & users.
- Also called resource allocator or manager.

⇒ OS services: There are 6 OS services!

① User Interface:

- Almost all OS has a user interface.
- It is a medium for the user to interact with OS.
- ⇒ There are 3 medium of communication:

① Command line interface: Line by line commands are passed to interact with OS.

② Batch interface: Multiple commands are executed at the same time.

③ Graphical User Interface: It tries to interact based on the location of the mouse pointer. We will have a graphical view.

① → Place where we write the commands are called interpreter goes line by line execution.

→ gets executed by user specified command.

→ Ex: rm file.txt (remove file command)

→ Sometimes implemented using kernel or by system program.

② → they are entered into a file.

→ these file are executed.

→ commands & directives control the system.

③ → provides mouse based windows & menu system as an interface

→ It has few icon & when mouse pointer is moved it either open it or other options.

→ It is user friendly, usually mouse, keyboard, monitor.

② Program Execution:

- System must be able to load program into memory, run the program, execution, either normally or abnormally (errors) & terminate.

③ Input/output operations:

- A running program may require I/O, which may involve a file or an I/O device.
- I/O devices have to be managed.

④ File System manipulation:

- File system is of particular interest.
- program needs to read & write files & directories.
- create & delete them.
- search them, list file information & permission management

⑤ Communication:

- processes may exchange information on the same computer or between computers over a network.
- Communication may be through:
 - ← shared memory
 - ← message passing

⑥ Error Detection:

- OS needs to be constantly aware of possible errors
- Errors can be in CPU or memory hardware, I/O devices, or in user program.
- For each type of error, OS should take appropriate action to ensure correct & consistent computing.
- Debugging facility can greatly enhance the user's & programmer's abilities to efficiently use the system.

* Services offered by System:

→ these are for ensuring efficient operation of the system.

① Resource Allocation:

- When multiple users or multiple jobs running concurrently, resources must be allocated to each of them.
- Different types of resources are connected to device & governed by OS.
- Some may have special allocation code Ex: CPU cycle, main memory etc.
- Some may have general request & release code.

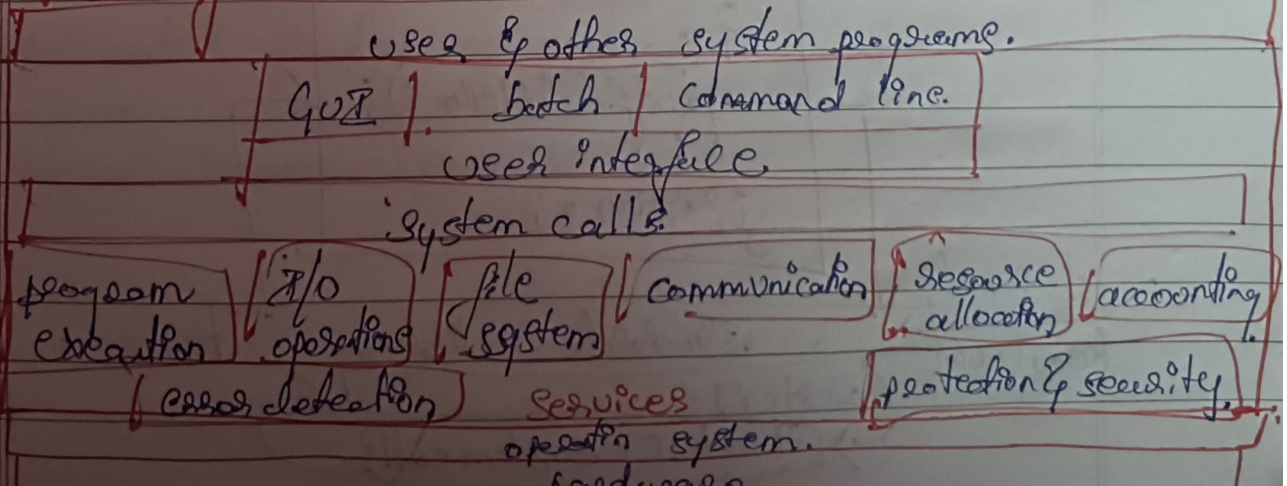
② Accounting:

- To keep track of which users use how much & what kinds of computer resources.

③ Protection & Security:

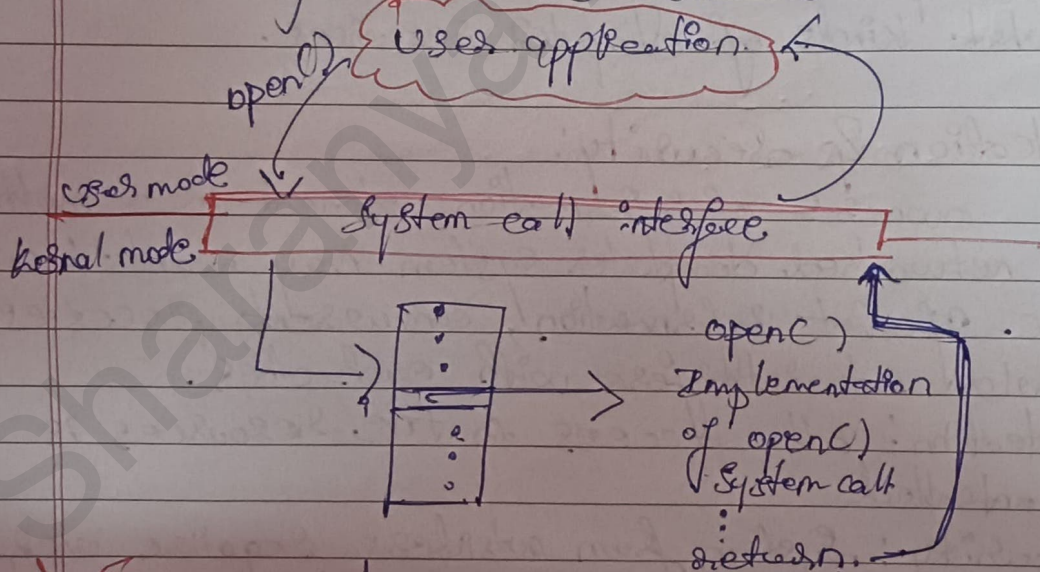
- The owners of information stored in a multiuser or networked computer system may want to control use of that information, concurrent processes should not interfere with each other.
- protection: all access to the resources is controlled
- Security: System from outsider's disguise user authentication, blocks invalid attempts.

⇒ View of OS services:

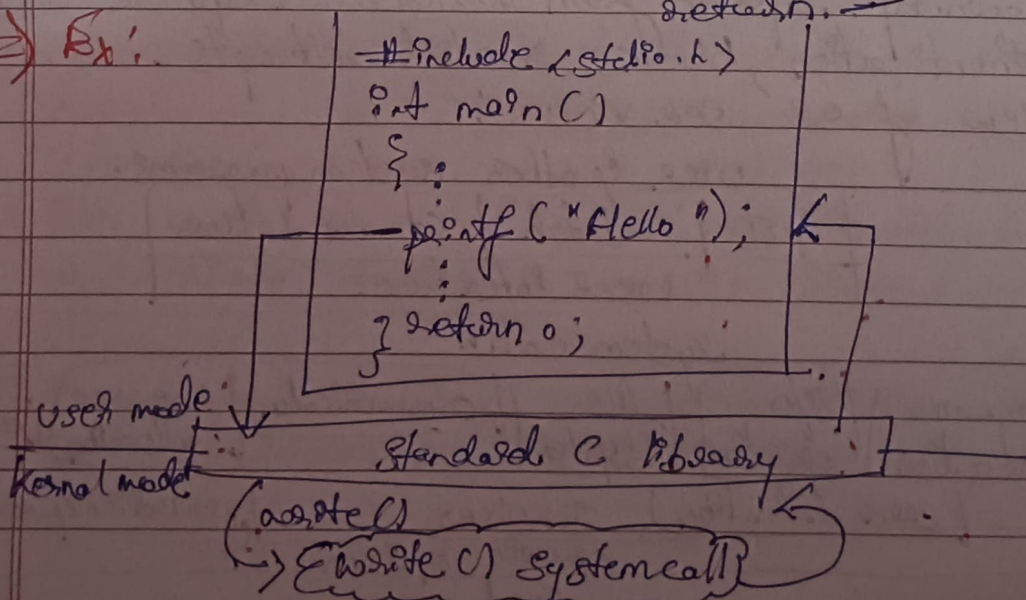


* System Call :

- It is a mechanism used by the program to request services from the OS.
- Program can interact with system hardware & data.
- When we request system call is initiated by program executing specific instructions you switch from user mode to kernel mode.
- OS handle request & provides a response to program.
- Every system call has number.
- System call sequence to copy the contents of one file to another file.
- System call interface maintains a table indexed according to these numbers.



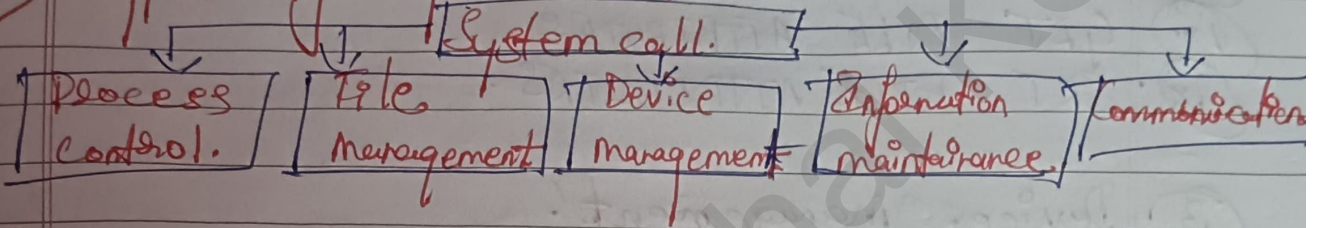
⇒ Ex:



→ 3 methods used to pass parameter to OS:

- (i) pass parameter in registers.
- (ii) parameter stored in block or table in memory, address of block passed as parameter in register.
- (iii) Parameter pushed onto stack by program & popped off the stack by the OS.

⇒ Types of System Call:



(i) Process Control System call: these include.

(i) END: closing the process normally.

(ii) ABORT: closing the process abnormally (with error).

(iii) LOAD: loading the process

(iv) EXECUTE: executing the process.

(v) CREATE: creating the process.

(vi) TERMINATE: Terminating the process.

(vii) GET: Getting the process attributes.

(viii) SET: Setting the process attributes

(ix) wait for time: It is the delay that is generated.

(x) Allocated & free memory:
 ↓
 ↳ reallocating the memory.
 Allocate memory of process

② File Management / Manipulation:

- i) CREATE FILE
- ii) DELETE FILE
- iii) OPEN FILE
- iv) CLOSE FILE
- v) READ
- vi) WRITE / UPDATE
- vii) REPOSITION
- viii) GET FILE ATTRIBUTES
- ix) SET FILE ATTRIBUTES

③ Device Management:

→ Deals with device drivers.

- i) Request Device
- ii) Release Device
- iii) Read write reposition
- iv) get & set device attribute
- v) logically attach / detach device

④ Information maintenance:

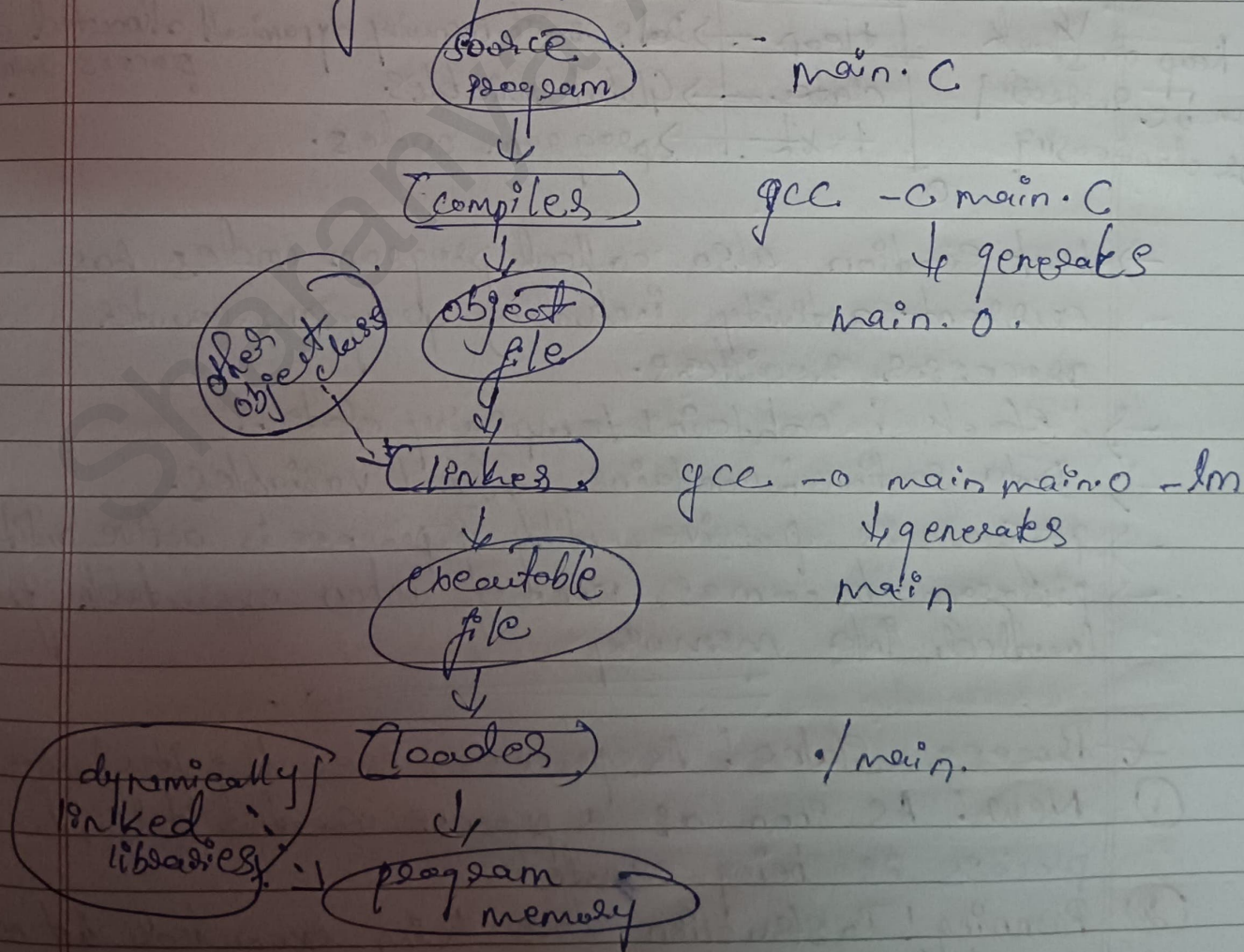
- GET TIME or DATE
- SET TIME or DATE
- GET SYSTEM DATA
- SET SYSTEM DATA
- GET process, file or device attributes
- SET process, file or device attributes

⑤ Communication:

- CREATE communication connection
- DELETE communication connection
- SEND information
- RECEIVE information
- ATTACH REMOTE DEVICE
- DETACH REMOTE DEVICE

* Role of Linker's & Loaders:

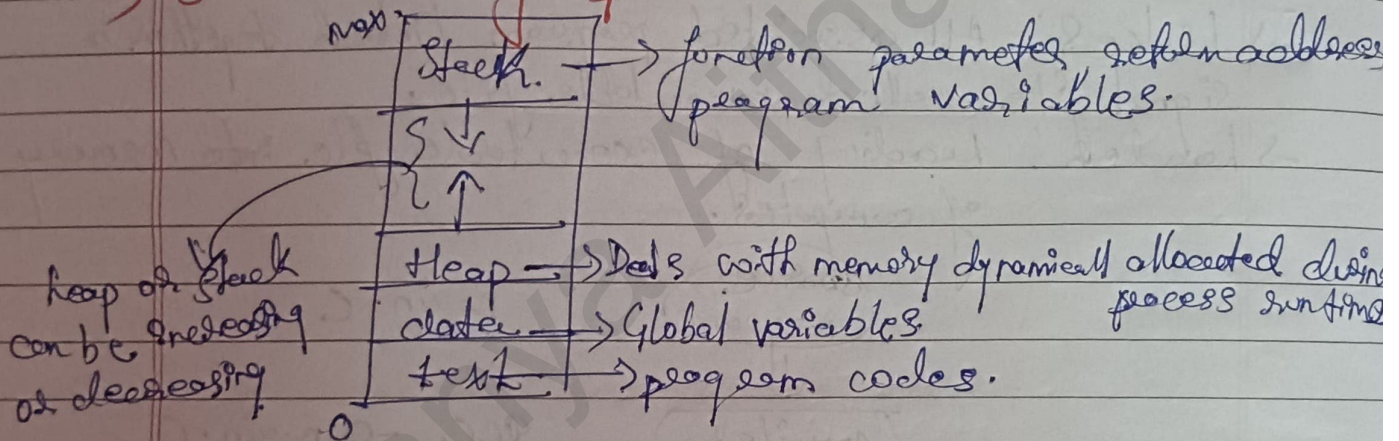
- Source code compiled into object files designed to be loaded into any physical memory location — relocatable object file.
- Linker combines these into single binary executable file, also brings in libraries.
- Program resides on secondary storage as binary executable, must be brought into memory by loader to be executed.
- Compiles: Source program to object program conversion.
- Linker: All associated program linked together a single file called executable file.
- Loader: Loading of executable file into memory of computer.



② PROCESS MANAGEMENT

- Program are set of instructions.
- Process: It is a program in execution which forms basis of all computation.
- It should progress in sequential fashion.
- Process includes:
 - program counter
 - Stack
 - data section

⇒ structure of process:



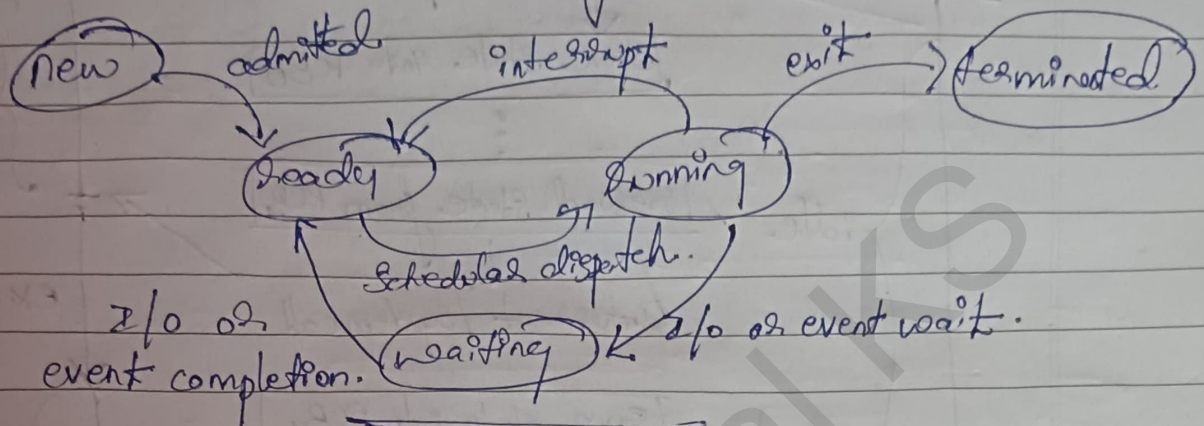
- Text section also called program codes has current activity including program counter, processor registers.
- Stack: contains temporary data.
- Data section: contains global variables.
- Program is passive entity & process is active entity.
- Program becomes process when executable file loaded into memory.

Process States: As process executes it changes its

- ① **New:** As soon as a process created it is new state process is being created.
- ② **Running:** Instructions are being executed but not completed.
- ③ **Waiting:** Process is waiting for some event to occur I/O device or any expectations.

(4) **Ready**: The process is waiting to be assigned by the processor.

(5) **Terminated**: Process has finished execution.



Process Control Block: (PCB)

Process state
process number
program counter
registers
memory limits
lists of open files
...

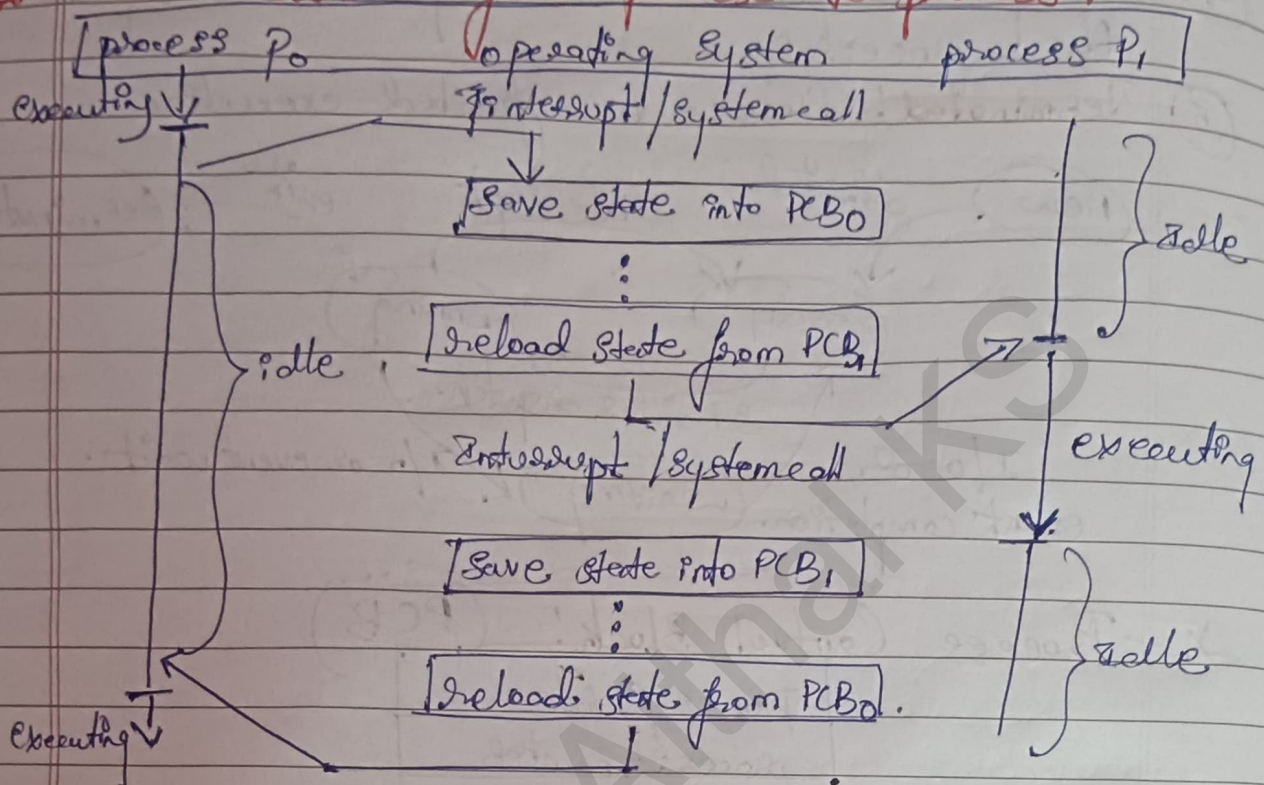
Information Associated with each process.

- (i) **Process state**: Current condition of process: ^{new, ready, running, waiting, terminated}
- (ii) **program counter**: Which is the next instruction to execute
- (iii) **CPU registers**: Small memory used by the CPU.
- (iv) **CPU scheduling information**: Time given for different programs.
- (v) **Memory management information**: Time taken for execution.
- (vi) **Accounting information**: Dynamically allocated memory.
- (vii) **I/O state's information**: list of I/O device a program uses.

Context Switching

(PCB)

CPU Switch from process to process:



Process Scheduling:

→ Used to!

- ① maximise CPU use, quickly
- ② quickly switch processes onto CPU for time sharing

Process Scheduler's:

→ It selects among available processes for next execution on CPU.

→ It maintains scheduling queues of process.

① Job queue:

→ Set of all processes in the system

→ As process enters system task need to be provided

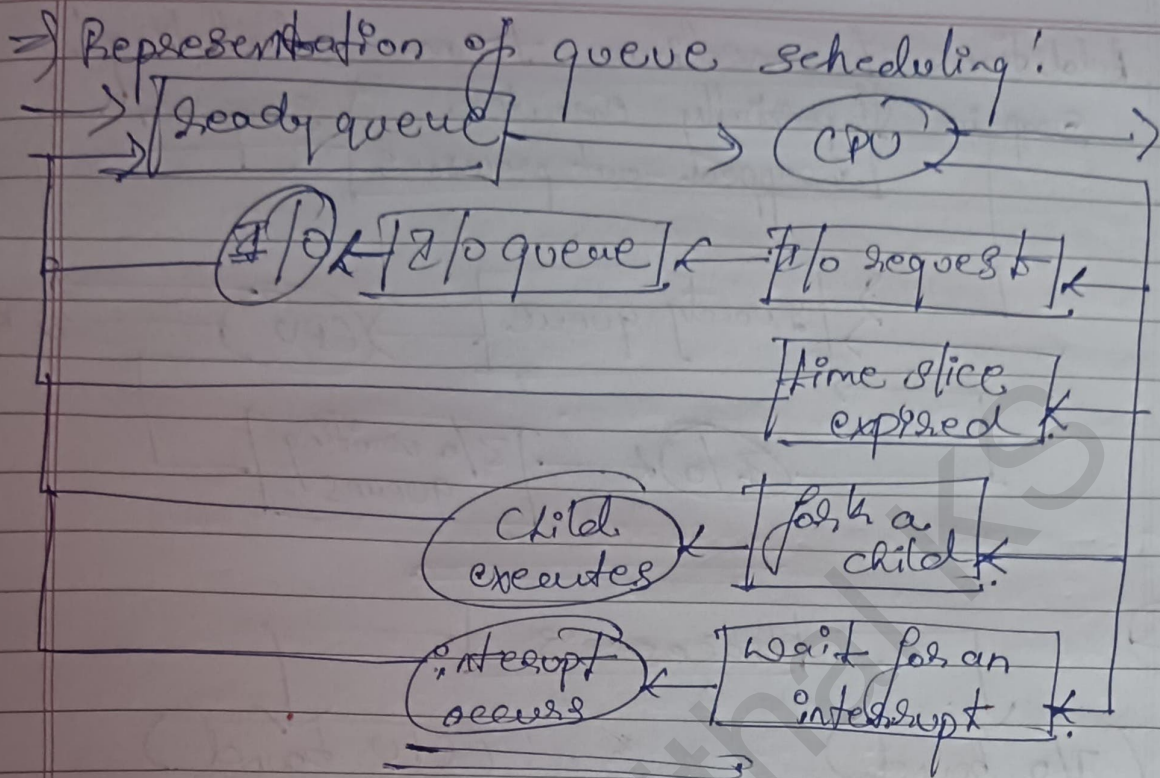
② Ready queue:

→ Set of all processes residing in main memory, ready & waiting to execute

③ Device I/O queue:

→ Set of processes waiting for I/O devices.

→ Every device will have its own device queue



* Schedulers:

- Process migrates among various scheduling process
- OS must select for scheduling purpose which process should get which queue.

2) Types:

① Long Term scheduler [Job scheduler]:

- Select which processes should be brought to ready queue from Job queue.
- Select process from pool & load them in main memory for execution.
- Invoked very infrequently.

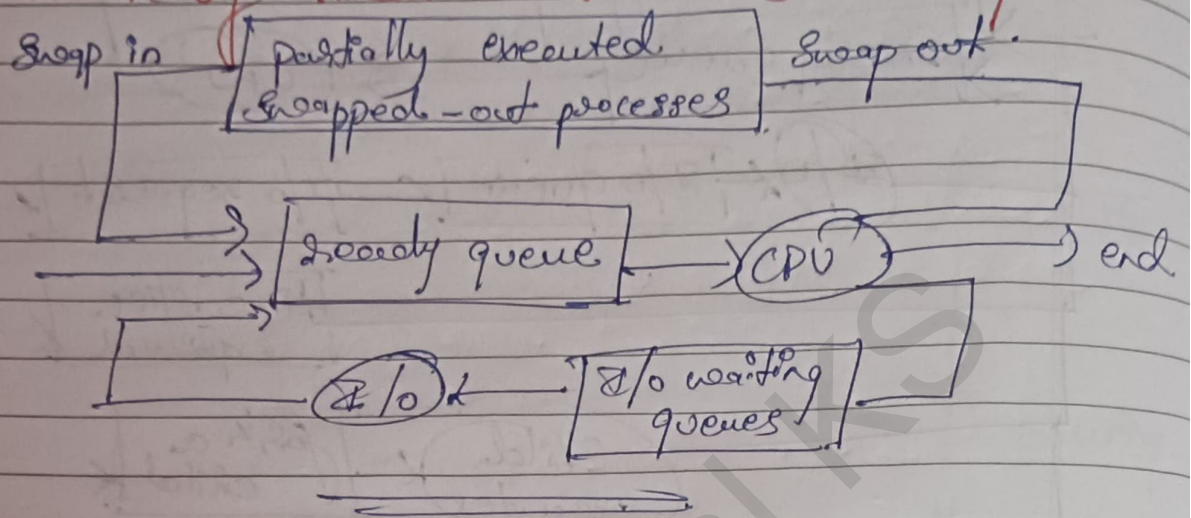
② Short term scheduler [CPU scheduler]:

- Select which process should be executed next & allocates CPU from ready queue.
- Sometimes only scheduler in system.
- Invoked very frequently.

③ Medium term scheduler:

- It removes some processes from memory.
- It reduces degree of multiprogramming.
- It avoids aggressive multitasking.
- Avoids performance degradation.

→ Addition of medium term scheduling!



(A)

Processes

(I/O bound)

(CPU bound)

- spends more time doing I/O computations
- many short CPU bursts

- more time doing CPU computations.
- few long CPU bursts.

* Interprocess Communication!

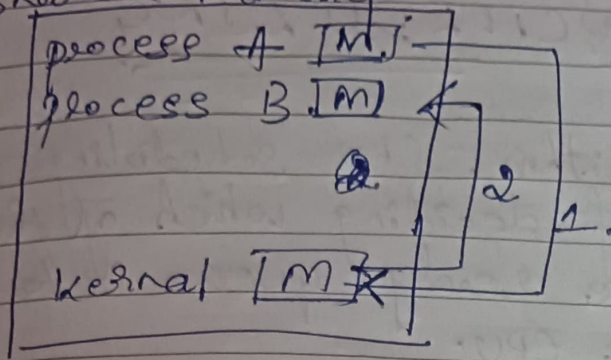
- process within system may be independent or cooperating.
- Independent: cannot be effected by the execution of another process
- Cooperating process can effect or be effected by other process, including sharing data.
- Reason for cooperating process:

- ① Information sharing.
- ② Computation speed.
- ③ modularity!
- ④ Convenience.

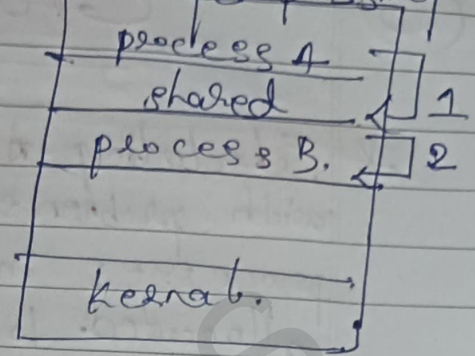
Advantages!

- they need interprocess communication (IPC)
- Two types of IPC
 - ① shared memory
 - ② message passing.

① Shared memory



② message passing



* Communication: $\rightarrow$ Direct
 $\rightarrow$ Indirect

* Synchronization:

message passing can be $\rightarrow$ blocking (synchronous)
 $\rightarrow$ non-blocking (asynchronous)

③ CPU SCHEDULING

* Scheduling Algorithm: CPU scheduling deals with problems of deciding which of the processes in the ready queue is to be allocated to the CPU.

* Scheduling Criteria:

① CPU Utilization:

- Keep the CPU as busy as possible.
- Conceptually CPU utilization is 0% to 100%.
- Real time it is 40% to 90%.

② Through put: It is number of processes that are completed per unit time.

③ Turn Around Time: Amount of time to execute a particular process.

- It is interval time of submission of process to time of completion of process.

$$TAT = CT - AT$$

CT: completion Time AT: Arrival time.

④ Burst time: Amount of time a process needs to utilize the CPU to complete its execution.

⑤ Waiting Time: Amount of time a process has been waiting in the ready queue.

$$WT = TAT - BT \quad BT: \text{Burst time.}$$

⑥ Response Time: The time from submission of request until the first response is produced CPU responded - AT.

* Scheduling Algorithm Optimization Criteria:

→ Maximum utilization of CPU, throughput,

→ Minimum TAT, WT & RT.

(FCFS)

① First Come First Serve Scheduling!

process	Priority	BT / CPU Execution.
P ₁	4	8
P ₂	1	6
P ₃	2	1
P ₄	2	9
P ₅	3	3

Gantt Chart :	P ₁	P ₂	P ₃	P ₄	P ₅	
	0	8	14	15	24	27

$$TAT = CT - AT \quad WT = TAT - BT$$

Process	TAT (milli sec)	WT (milli sec)
P ₁	8 (8-0)	0 (8-8)
P ₂	14 (14-0)	8 (14-6)
P ₃	15 (15-0)	14 (15-1)
P ₄	24 (24-0)	15 (24-9)
P ₅	27 (27-0)	24 (27-3)

Arrival time not given : It is Zero.

AVG ~~WT~~ TAT :

$$= \frac{8 + 14 + 15 + 24 + 27}{5}$$

$$= \frac{61}{5} = 12.2 \text{ milli sec.}$$

AVG TAT :

$$= \frac{0 + 8 + 14 + 15 + 24}{5}$$

$$= \frac{58}{5} = 11.6 \text{ milli sec.}$$

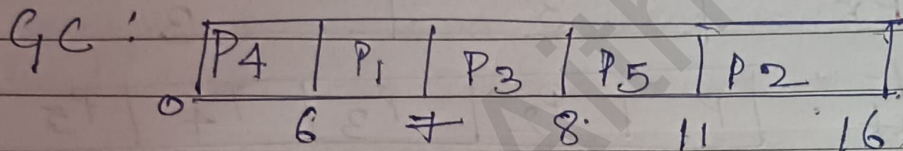
② Shortest Job First Scheduling (SJFS):

SJFS $\left\{ \begin{array}{l} \rightarrow \text{Primitive: can be stopped} \\ \rightarrow \text{Non primitive: cannot be stopped} \end{array} \right.$

process	AT	BT	TAT	WT
P ₁	2	1		
P ₂	1	5		
P ₃	4	1		
P ₄	0	6	5	4
P ₅	2	3	2	

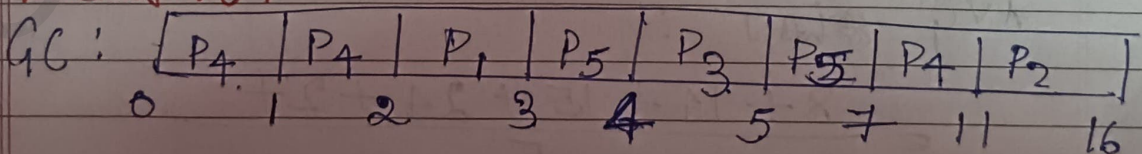
We check BT then AT.

(X) Non-Primitive:



- First we take P₄ even though BT of P₄ is 3 which is not shortest as the AT of P₄ is 0.
- Next we completely follow SJF rule i.e. just see BT as 6 is BT for P₄ the maximum. BT reached & all process already occurred to ready queue.
- So we go in BT shortest order $P_1 \rightarrow P_3 \rightarrow P_5 \rightarrow P_2$

(X) Primitive:



- First we take P₄ as AT for P₄ is 0 it will execute for 1 ms then P₂ arrives. But we continue P₄ for other 1 ms as BT for P₄ is also 5 & P₂ is also 5 when P₂ arrived.
- P₁ is executed next at 2 ms to 3 ms i.e. where P₅ occurs & will execute till 4 ms until P₃ arrives & executes for 1 ms. till now BT is 5 ms of total.

→ Now we go according to ~~STF~~ F as all job occurred so we choose $P_5 \rightarrow P_4 \rightarrow P_2$

GC:	P4	P1	P5	P3	P5	P4	P2	
	0	2	3	4	5	7	11	16

③ Priority Scheduling:

PS $\begin{cases} \rightarrow \text{Preemptive} \\ \rightarrow \text{Non preemptive} \end{cases}$

process	AT	Priority	BT	TAT	WT
P1	0	3	8 3		
P2	1	4	2		
P3	3	4	4		
P4	4	5	1		
P5	5	2	6 1		
P6	6	6	5		
P7	10	1	1		

④ Non preemptive:

GC:	P ₁	P ₅	P ₇	P ₂	P ₃	P ₄	P ₆	
	0	8	14	15	17	21	22	27

- First considered P1 as AT is 0. Run for 8ms.
- Now after 8ms accept p7 all other occurred so go by priority to them.
- we do p5 which runs for 1ms when p7 also occurred which has priority 1 so execute it first.
- Rest go by priority $P_2 \rightarrow P_3 \rightarrow P_4 \rightarrow P_6$.

⑤ Preemptive:

Primitive:										
GC:	P ₁	P ₅	P ₇	P ₅	P ₁	P ₂	P ₃	P ₄	P ₆	
	0	5	10	11	12	15	17	21	22	27

④ Round Robin Scheduling : [RRS]

~~RRS~~ ~~Preemptive~~
Non-Preemptive.

Time Quantum = 3 ms.

Process	AT	BT	TAT	WT
P ₁	5	2		
P ₂	4	3		
P ₃	3	4		
P ₄	1	3		
P ₅	2	2		
P ₆	6	3		

→ NOTE: CPU starts with 0. Then BT = Rnd GC else not equal.

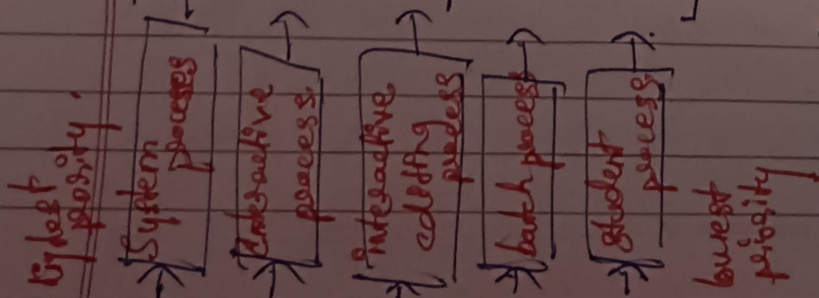
RR: ~~P₄~~ ~~P₅~~ ~~P₃~~ ~~P₂~~ ~~P₄~~ ~~P₁~~ ~~P₆~~ ~~P₃~~ ~~P₂~~ ~~P₄~~ ~~P₁~~ ~~P₃~~

GC: ~~P₄~~ P₅ P₃ P₂ P₄ P₁ P₆ P₃ P₂ P₄ P₁ P₃
0 1 4 6 9 12 15 18 21 24 27 30 32 33

Here BT = 32 but GC last = 33 as.
from 0 to 1ms no process at all.

* Multilevel Queue Scheduling:

- Another class of Scheduling Algorithm.
- Where process are easily classified into different group.
- Ex: Interactive foreground process,
Batch background process.
- Here Ready queue is partitioned into several queue
- process are permanently assigned to particular queue



④

THREADS

Threads are fundamental unit of CPU utilization that forms the basis of multithreaded computer systems.

- Thread creation is light weighted.
- It simplifies code.
- Increases Efficiency.

⇒ Benifits:

① Responsiveness: Multithreading may allow a program to continue running if even part of it is blocked or is performing a lengthy operation. There by increasing responsiveness to users.

② Economy: Allocating memory & resources for a process creation is costly, because threads share resources of the process to which they belong, it is more economical to create and context switch threads.

③ Resource Sharing: By default threads share the memory & resources of the process to which they belong several different threads in an application share same address space.

④ Scalability: Benifit of multithreading can be greatly increased in multiprocesses, architecture where threads may be running in parallel on different processes.

Single

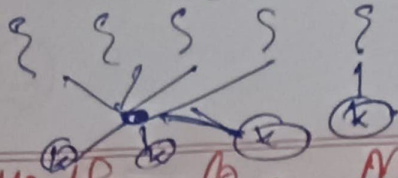
code	data	files
Register	stack	

→ (Thread)

Multithreaded

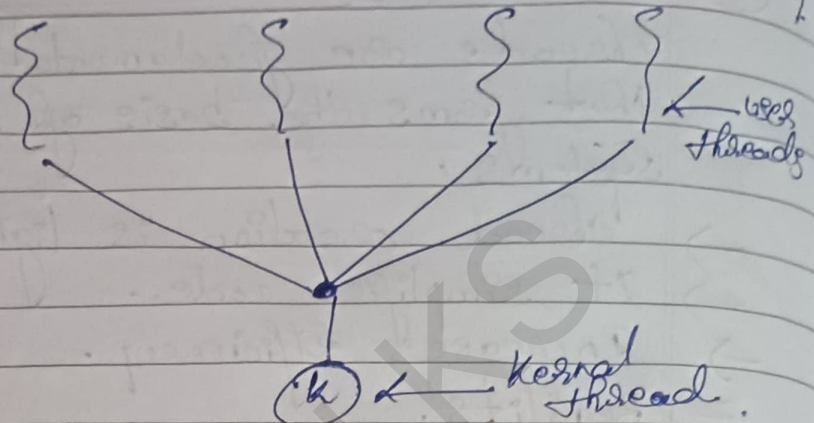
code	data	files
Register	Register	Register
stack	stack	stack

④ 2 level model! Combination of many-many ^{classmate} to 1



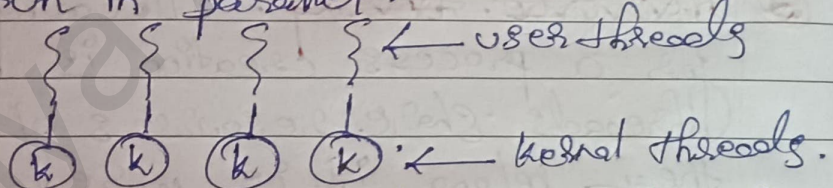
Multi threading Models: 1-1 / many-1 / many-many

① many to 1:



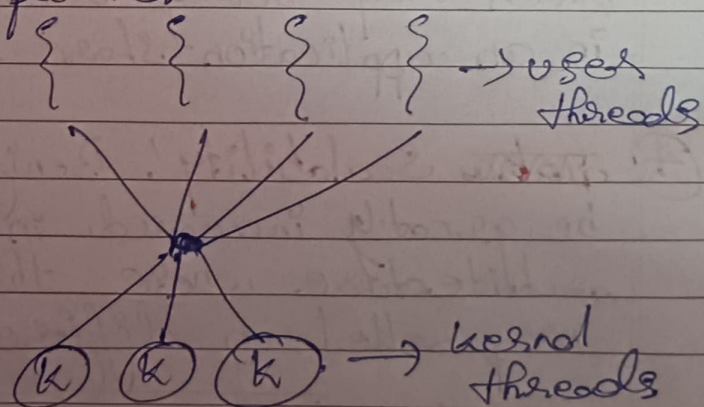
- Maps many user threads to 1 kernel level thread.
- It is efficient.
- In case one thread makes block system call the entire process gets block.
- Only one thread can access kernel at a time.
- No threads run in parallel.

② 1 to 1:



- Maps each user threads to a kernel threads.
- provides more concurrency than many to 1 by allowing another thread to run.
- When one thread makes blocking system other threads run in parallel.

③ many to many:



- Multiplexes many user level threads to smaller or equal number of kernel threads.
- provides developers to create as many as user thread he wants.
- kernel schedule only one thread at a time.